Group 18 – Class Design Specification

1. Design for class “Cart”

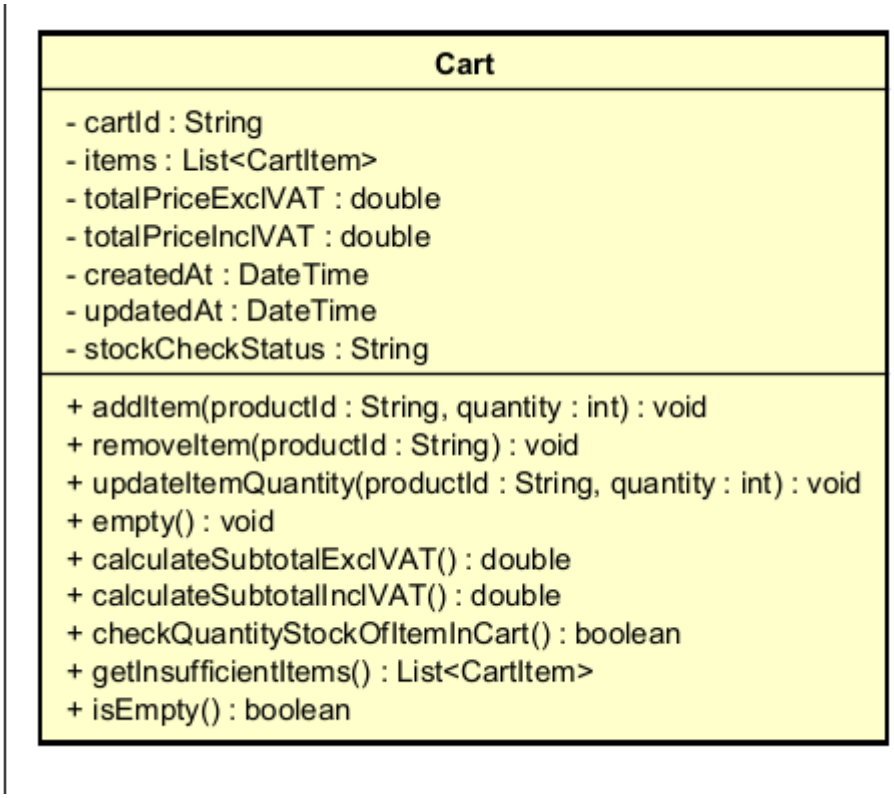


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cartId	String	null	Cart ID
2	items	List	[]	List of items in the cart
3	totalPriceExclVAT	double	0.0	Total price excluding VAT
4	totalPriceInclVAT	double	0.0	Total price including VAT
5	createdAt	DateTime	now()	Cart creation time
6	updatedAt	DateTime	now()	Last updated time
7	stockCheckStatus	String	"UNCHECKED"	Stock checking status

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	addItem(productId:String, quantity:int)	void	Add a product to the cart
2	removeItem(productId:String)	void	Remove a product from the cart
3	updateItemQuantity(productId:String, quantity:int)	void	Update item quantity
4	empty()	void	Clear the entire cart
5	calculateSubtotalExclVAT()	double	Calculate total price excluding VAT
6	calculateSubtotalInclVAT()	double	Calculate total price including VAT
7	checkQuantityStockOfItemInCart()	boolean	Check stock availability for each item
8	getInsufficientItems()	List	Get list of items with insufficient stock
9	isEmpty()	boolean	Check if the cart is empty

Parameters :

- productId: product identifier
- quantity: desired quantity

Exceptions:

- InvalidQuantityException: if quantity ≤ 0
- ProductNotFoundException: if the product does not exist
- InsufficientStockException: if requested quantity exceeds available stock

Methods:

- addItem(): Adds a new item or increases quantity if the item already exists in the cart.
- checkQuantityStockOfItemInCart(): Iterates through all CartItems and compares requested quantity with available stock.
- calculateSubtotalInclVAT(): Adds 10% VAT to item prices according to the problem requirement.

2. Design for class “CartItem”

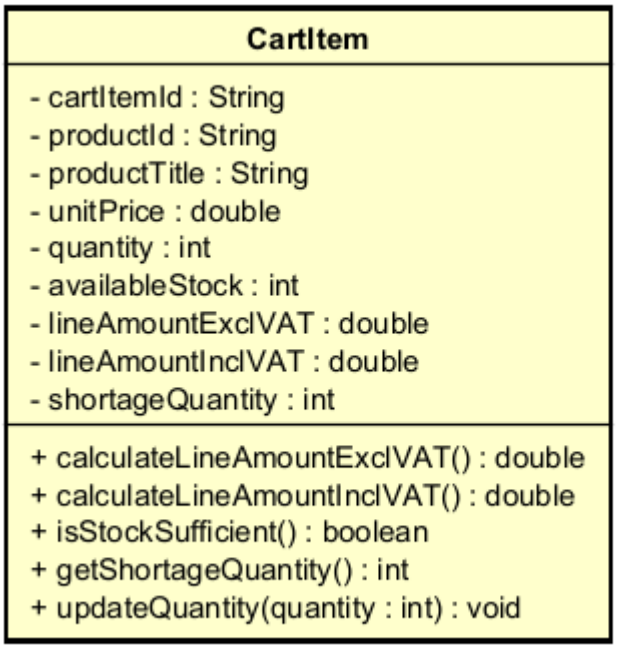


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cartItemId	String	null	Cart item ID
2	productId	String	null	Product ID
3	productTitle	String	""	Product title
4	unitPrice	double	0.0	Unit price excluding VAT
5	quantity	int	1	Selected quantity
6	availableStock	int	0	Current available stock
7	lineAmountExclVAT	double	0.0	Line amount excluding VAT
8	lineAmountInclVAT	double	0.0	Line amount including VAT
9	shortageQuantity	int	0	Quantity shortage if stock is insufficient

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	calculateLineAmountExclVAT()	double	Calculate line amount excluding VAT
2	calculateLineAmountInclVAT()	double	Calculate line amount including VAT
3	isStockSufficient()	boolean	Check if stock is sufficient
4	getShortageQuantity()	int	Return shortage quantity
5	updateQuantity(quantity:int)	void	Update item quantity

Exception

- InvalidQuantityException: if the quantity is invalid

3. Design for class “DeliveryInfo”

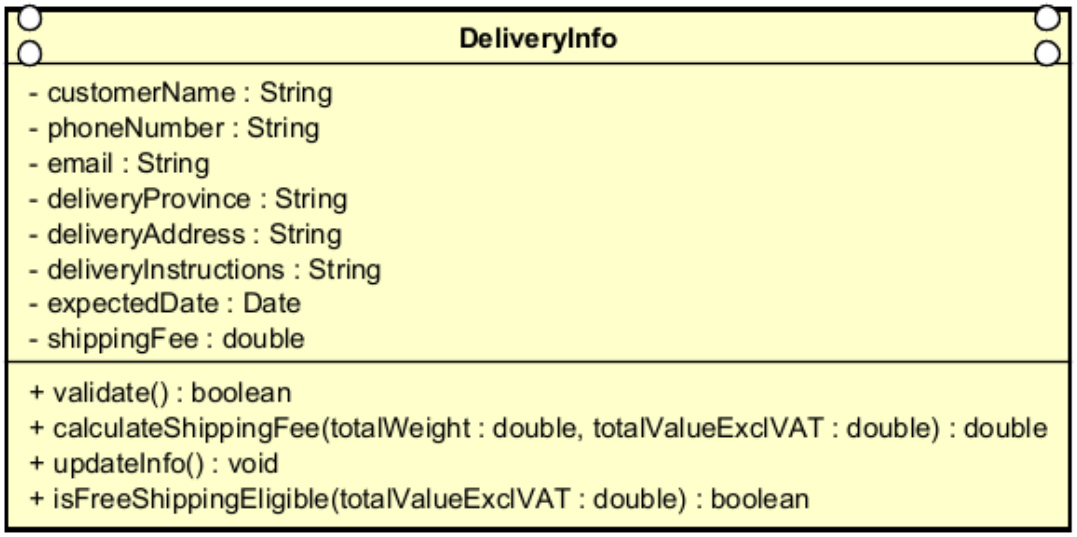


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	customerName	String	""	Receiver's name

2	phoneNumber	String	""	Phone number
3	email	String	""	Email for receiving invoice
4	deliveryProvince	String	""	Province/City
5	deliveryAddress	String	""	Delivery address
6	deliveryInstructions	String	""	Delivery instructions/notes
7	expectedDate	Date	null	Expected delivery date
8	shippingFee	double	0.0	Current shipping fee

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validate()	boolean	Validate delivery information
2	calculateShippingFee(totalWeight:double, totalValueExclVAT:double)	double	Calculate shipping fee
3	updateInfo(...)	void	Update delivery information
4	isFreeShippingEligible(totalValueExclVAT:double)	boolean	Update delivery information

Parameters

- totalWeight: total weight of the order
- totalValueExclVAT: total order value excluding VAT

Exceptions

- InvalidDeliveryInfoException: if required fields are missing or invalid format
- InvalidProvinceException: if the province/city is invalid

Methods

- validate(): checks customer name, phone number, email, address, and province/city.
- calculateShippingFee() follows the problem requirements:
- Hanoi / Ho Chi Minh City: 22,000 VND for the first 3 kg
- Other provinces: 30,000 VND for the first 0.5 kg
- Additional 2,500 VND for every next 0.5 kg
- Free shipping up to 25,000 VND if total item value > 100,000 VND

4. Design for class “Invoice”

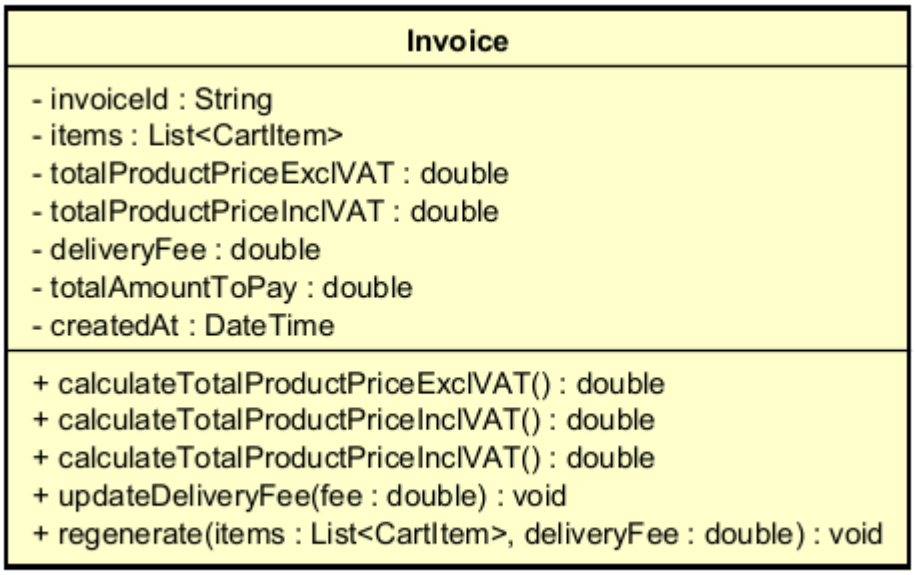


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	invoiceId	String	null	Invoice ID
2	items	List	[]	List of items
3	totalProductPriceExclVAT	double	0.0	Total product price excluding VAT
4	totalProductPriceInclVAT	double	0.0	Total product price including VAT
5	deliveryFee	double	0.0	Shipping fee
6	totalAmountToPay	double	0.0	Total amount to pay
7	createdAt	DateTime	now()	Invoice creation time

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	calculateTotalProductPriceExclVAT()	double	Calculate total product price excluding VAT

2	calculateTotalProductPriceInclVAT()	double	Calculate total product price including VAT
3	calculateTotalAmountToPay()	double	Calculate final payable amount
4	updateDeliveryFee(fee:double)	void	Update shipping fee
5	regenerate(items:List, deliveryFee:double)	void	Regenerate invoice when cart or delivery changes

Method

- totalAmountToPay = totalProductPriceInclVAT + deliveryFee
- Shipping fee is not subject to VAT, as specified in the requirements.

5. Design for class “Order”

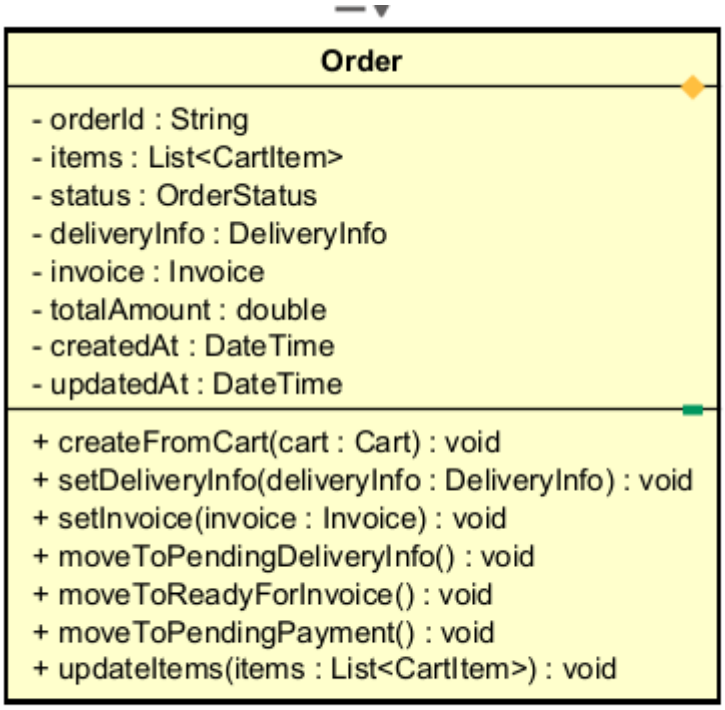


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	String	null	Order ID
2	items	List	[]	List of ordered items
3	status	OrderStatus	DRAFT	Order status
4	deliveryInfo	DeliveryInfo	null	Delivery information
5	invoice	Invoice	null	Current invoice
6	totalAmount	double	0.0	Total order amount
7	createdAt	DateTime	now()	Creation time
8	updatedAt	DateTime	now()	Last updated time

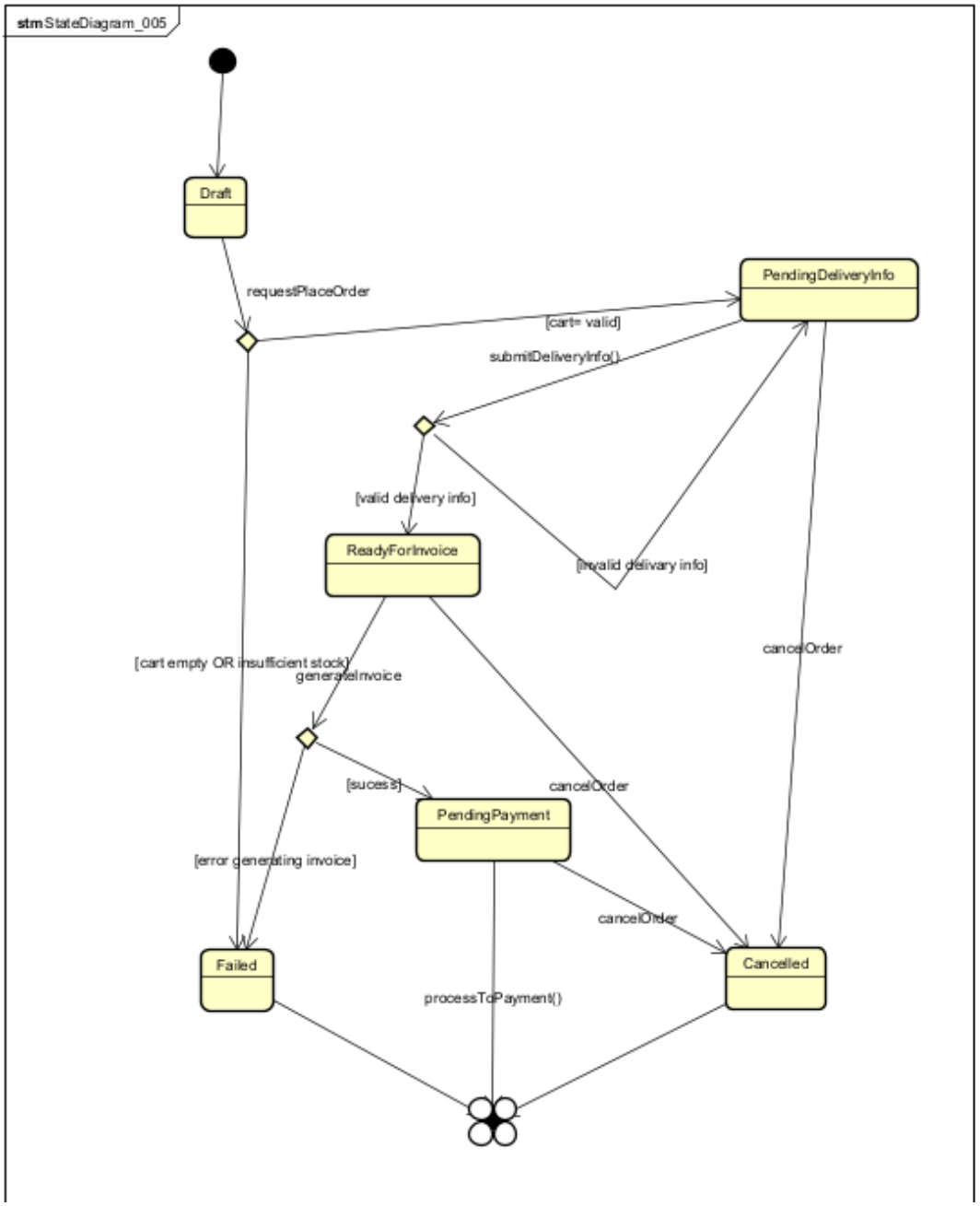
Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	createFromCart(cart:Cart)	void	Initialize order from cart
2	setDeliveryInfo(deliveryInfo:DeliveryInfo)	void	Attach delivery information
3	setInvoice(invoice:Invoice)	void	Attach invoice
4	moveToPendingDeliveryInfo()	void	Change state to pending delivery info
5	moveToReadyForInvoice()	void	Change state to ready for invoice
6	moveToPendingPayment()	void	Change state to pending payment
7	updateItems(items:List)	void	Update order items

Exception

- InvalidOrderStateException: if an invalid state transition occurs

State machine diagram



6. Design for class “CartService”

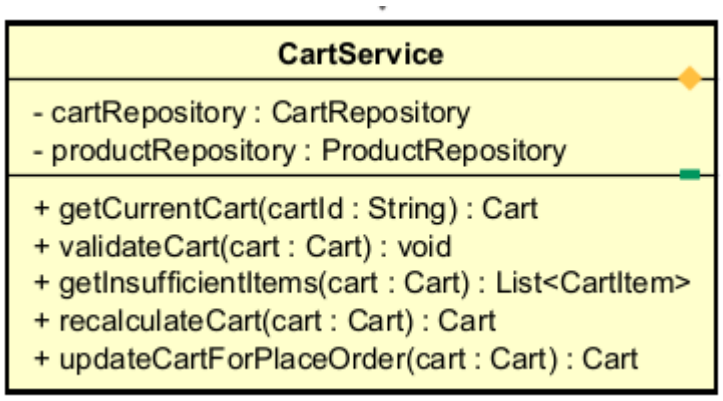


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cartRepository	CartRepository	injected	Access to cart data
2	productRepository	ProductRepository	injected	Access to product stock data

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getCurrentCart(cartId:String)	Cart	Retrieve the current cart
2	validateCartStock(cart:Cart)	boolean	Validate stock for the entire cart
3	getInsufficientItems(cart:Cart)	List	Get list of items with insufficient stock
4	recalculateCart(cart:Cart)	Cart	Recalculate cart subtotal
5	updateCartForPlaceOrder(cart:Cart)	Cart	Normalize/prepare cart before creating order

Methods

- validateCartStock() calls cart.checkQuantityStockOfItemInCart()
- recalculateCart() recalculates prices after quantity changes

7. Design for class “DeliveryService”

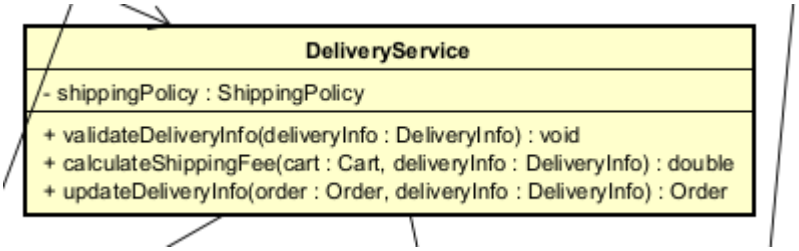


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	shippingPolicy	ShippingPolicy	injected	Shipping fee calculation policy

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateDeliveryInfo(deliveryInfo:DeliveryInfo)	void	Validate delivery information
2	calculateShippingFee(cart:Cart, deliveryInfo:DeliveryInfo)	double	Calculate shipping fee
3	updateDeliveryInfo(order:Order, deliveryInfo:DeliveryInfo)	Order	Attach/update delivery information for the order

Exception

- InvalidDeliveryInfoException if the delivery information is invalid

Method

- Calculate total weight from CartItem
- Use total value excluding VAT to check the free shipping rule

8. Design for class “InvoiceService”

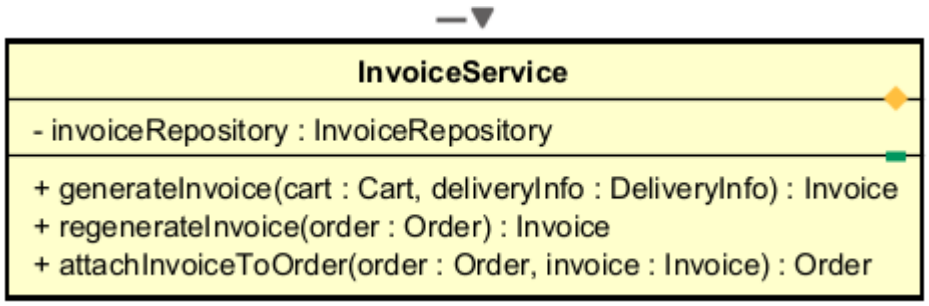


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	invoiceRepository	InvoiceRepository	injected	Access invoices

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	generateInvoice(cart:Cart, deliveryInfo:DeliveryInfo)	Invoice	Create an invoice from cart and delivery information
2	regenerateInvoice(order:Order)	Invoice	Regenerate an invoice when cart or delivery changes
3	attachInvoiceToOrder(order:Order, invoice:Invoice)	Order	Attach an invoice to an order

Method

- generateInvoice() calculates:
- Total product amount excluding VAT
- Total product amount including VAT
- Shipping fee
- Total amount to pay

9. Design for class “PlaceOrderControl”

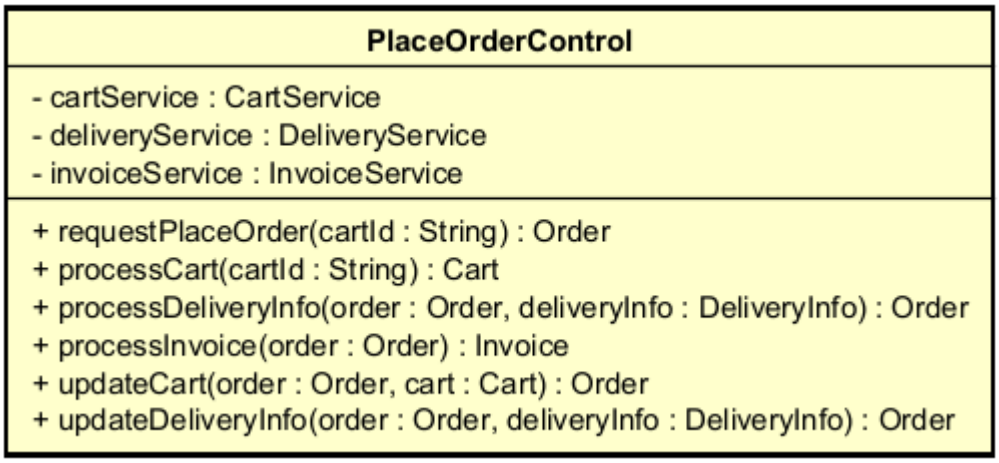


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cartService	CartService	injected	Service for handling cart
2	deliveryService	DeliveryService	injected	Service for handling delivery
3	invoiceService	InvoiceService	injected	Service for generating invoices

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestPlaceOrder(cartId:String)	Order	Start the place order use case
2	processCart(cartId:String)	Cart	Validate and normalize the cart
3	processDeliveryInfo(order:Order, deliveryInfo:DeliveryInfo)	Order	Process delivery information
4	processInvoice(order:Order)	Invoice	Generate a temporary invoice
5	updateCart(order:Order, cart:Cart)	Order	Update the order when the cart changes

6	updateDeliveryInfo(order:Order, deliveryInfo:DeliveryInfo)	Order	Cập nhật lại delivery + Update delivery info and invoice

Parameter

- cartId: Cart identifier
- order: The order being processed
- deliveryInfo: New delivery information

Exception

- InsufficientStockException: Thrown when items in the cart are out of stock
- InvalidDeliveryInfoException: Thrown when delivery information is invalid
- EmptyCartException: Thrown when the cart is empty

Method

- requestPlaceOrder()
- Load current cart
- Check if cart is empty
- Validate stock availability
- Create order from cart
- Proceed to input delivery information
- processDeliveryInfo()
- Validate delivery information
- Calculate shipping fee
- Update order
- processInvoice()
- Generate invoice
- Attach invoice to order
- Move order to PENDING_PAYMENT status

10. Design for class “CartScreen”

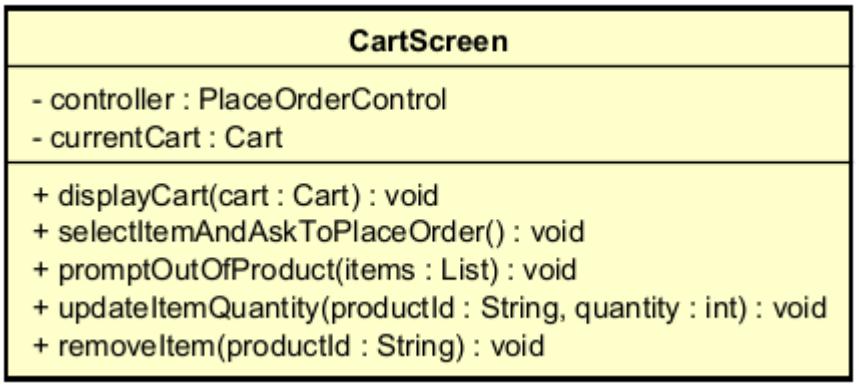


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	controller	PlaceOrderControl	injected	Controller place order
2	currentCart	Cart	null	Currently displayed cart

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayCart(cart:Cart)	void	Display the cart
2	selectItemAndAskToPlaceOrder()	void	Customer confirms to start placing an order
3	promptOutOfProduct(insufficientItems:List)	void	Notify items that are out of stock
4	updateItemQuantity(productId:String, quantity:int)	void	Allow updating item quantity
5	removeItem(productId:String)	void	Allow removing an item from the cart

11. Design for class “DeliveryForm”

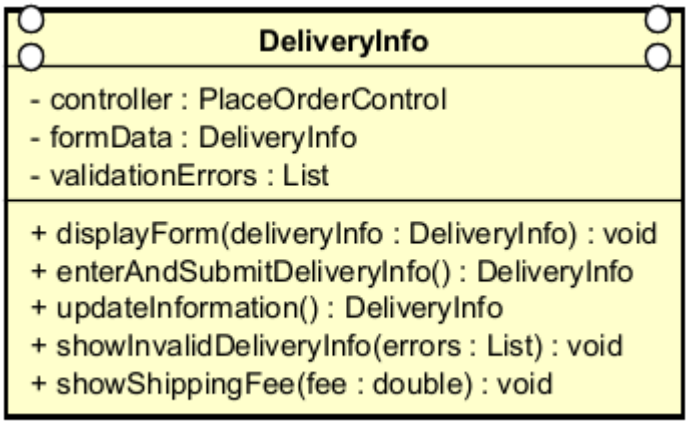


Table 1. Example of attribute design

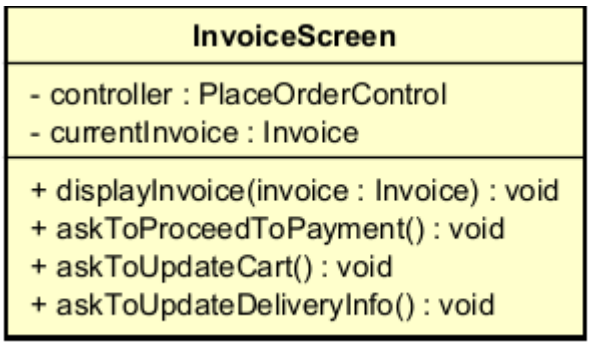
#	Name	Data type	Default value	Description
1	controller	PlaceOrderControl	injected	Controller place order
2	formData	DeliveryInfo	empty	Delivery form data
3	validationErrors	List	[]	List of validation errors

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayForm(deliveryInfo:DeliveryInfo)	void	Display the form
2	enterAndSubmitDeliveryInfo()	DeliveryInfo	Enter and submit delivery information
3	updateInformation()	DeliveryInfo	Update the information
4	showInvalidDeliveryInfo(errors:List)	void	Show input validation errors
5	showShippingFee(fee:double)	void	Display shipping fee after calculation

-

12. Design for class “InvoiceScreen”



- **Table 1. Example of attribute design**

#	Name	Data type	Default value	Description
1	controller	PlaceOrderControl	injected	Place order controller
2	currentInvoice	Invoice	null	Currently displayed invoice

- **Table 2. Example of operation design**

#	Name	Return type	Description (purpose)
1	displayInvoice(invoice:Invoice)	void	Display temporary invoice
2	askToProceedToPayment()	void	Yêu cầu sang bước thanh toán
3	askToUpdateCart()	void	Go back to update cart
4	askToUpdateDeliveryInfo()	void	Go back to update delivery information

-

13. Design for class “ProductDetailScreen”

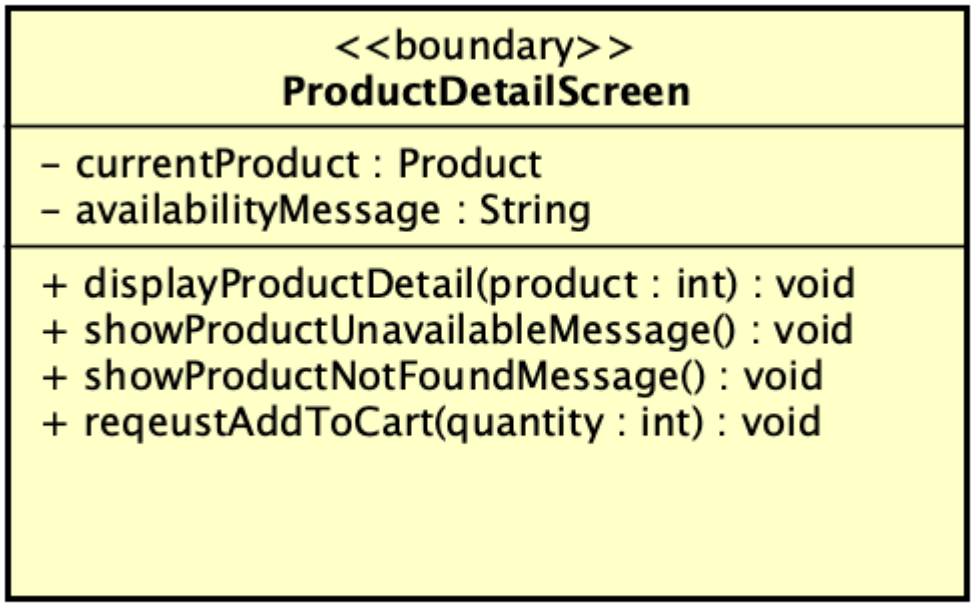


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	currentProduct	Product	null	Product currently displayed
2	availabilityMessage	String	“”	Availability notification
3	generalInfo	String	“”	General product information
4	specificInfo	String	“”	Type-specific product information

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayProductDetail(product: Product)	void	Display full product information
2	showProductUnavailableMessage()	void	Show unavailable notification
3	showProductNotFoundMessage()	void	Show product not found error
4	requestAddToCart(quantity : int)	void	Request adding product to cart

Parameter:

- product: product to display; must not be null.
- quantity: number of items to add; must be > 0.

Exception:

- IllegalArgumentException if product is null.
- InvalidQuantityException if quantity ≤ 0.

Method

- displayProductDetail() extracts general and specific info and renders UI.
- showProductUnavailableMessage() displays availability warning.
- showProductNotFoundMessage() displays error.
- requestAddToCart() triggers Add-to-Cart use case.

Other diagrams

- Optional sequence diagram.

14. Design for class “ProductListScreen”

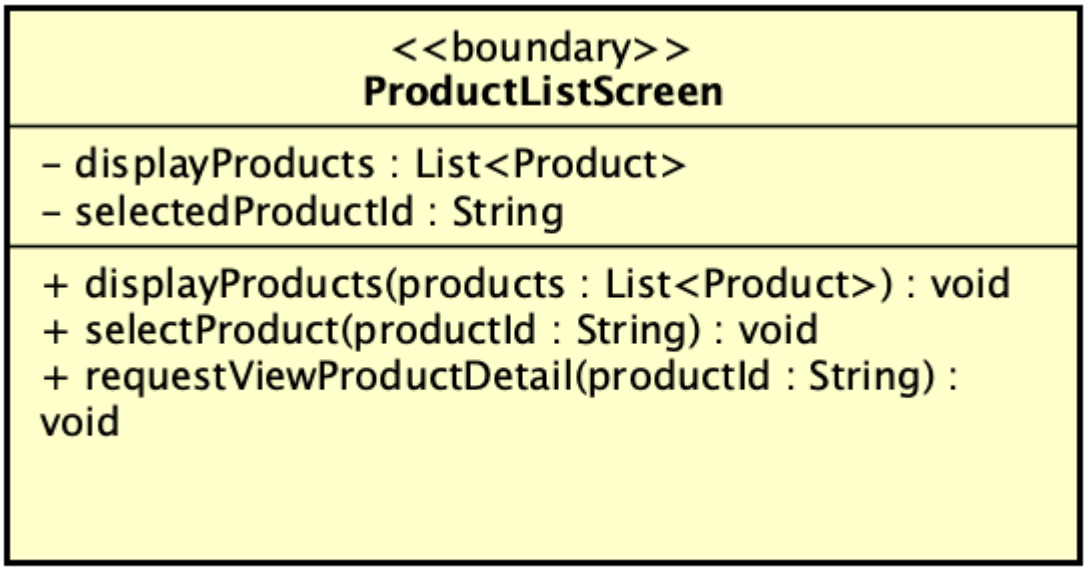


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	displayedProducts	List<Product>	empty list	List of products currently displayed on screen
2	selectedProductID	String	null	ID of the product selected by the

				customer
3	searchKeyword	String	“”	Current search keyword
4	filterCriteria	String	“”	Current filtering condition

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayProducts(products:List<Product>)	void	Display a list of products on the screen
2	selectProduct(productID: String)	void	Record the selected product

Parameter:

- products: list of products to display; default empty.
- productId: selected product ID; must be valid.

Exception:

- IllegalArgumentException if productId is null or empty
- ProductNotInDisplayedListException if not found in displayed list

Method

- displayProducts() updates displayedProducts and renders UI.
- selectProduct() updates selectedProductId.
- requestViewProductDetail() sends request to controller.

15. Design for class “ViewProductDetailController”

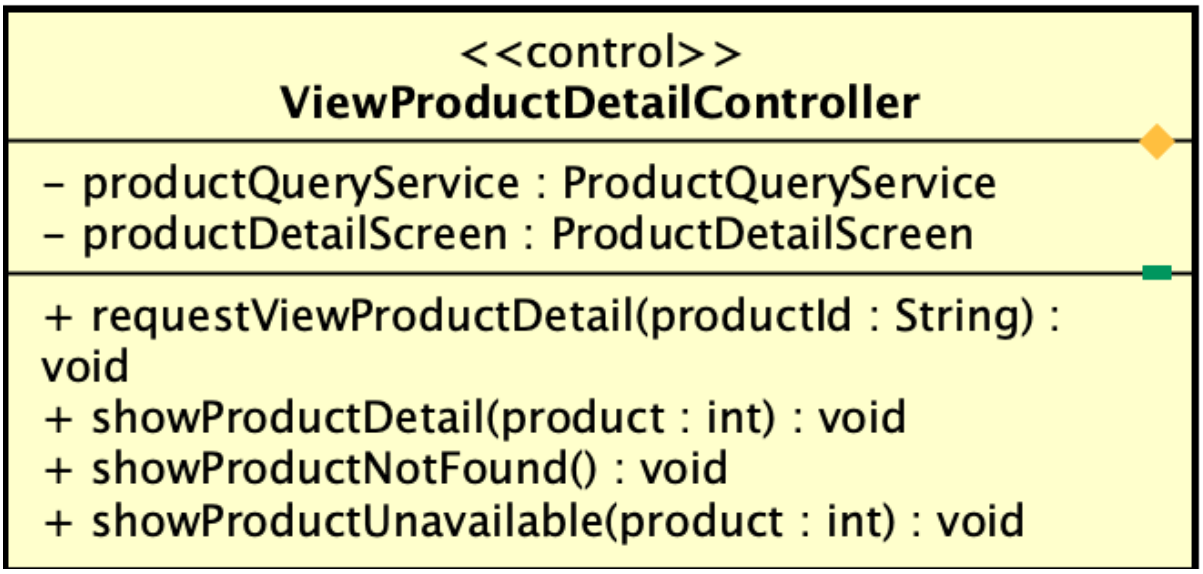


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productService	ProductService	null	Service to retrieve product data
2	productDetailScreen	ProductDetailScreen	null	UI screen for displaying details

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestViewProductDetail(productId: String)	void	Handle request from UI
2	showProductDetail(product : Product)	void	Send product data to UI
3	showProductNotFound()	void	Display not found error
4	showProductUnavailable(product: Product)	void	Display unavailable status

Parameter:

- productId: ID of product
- product: retrieved product object.
-

Exception:

- IllegalArgumentException if productId invalid.

- SystemDataAccessException if retrieval fails.

Method

- requestViewProductDetail():
1. Validate productId
2. Call service
3. Handle null → not found
4. Check availability
5. Display result
- showProductDetail() updates UI.
- showProductUnavailable() displays product + warning.

Other diagrams

- showProductUnavailable() displays product + warning.

16. Design for class “ProductQueryService”

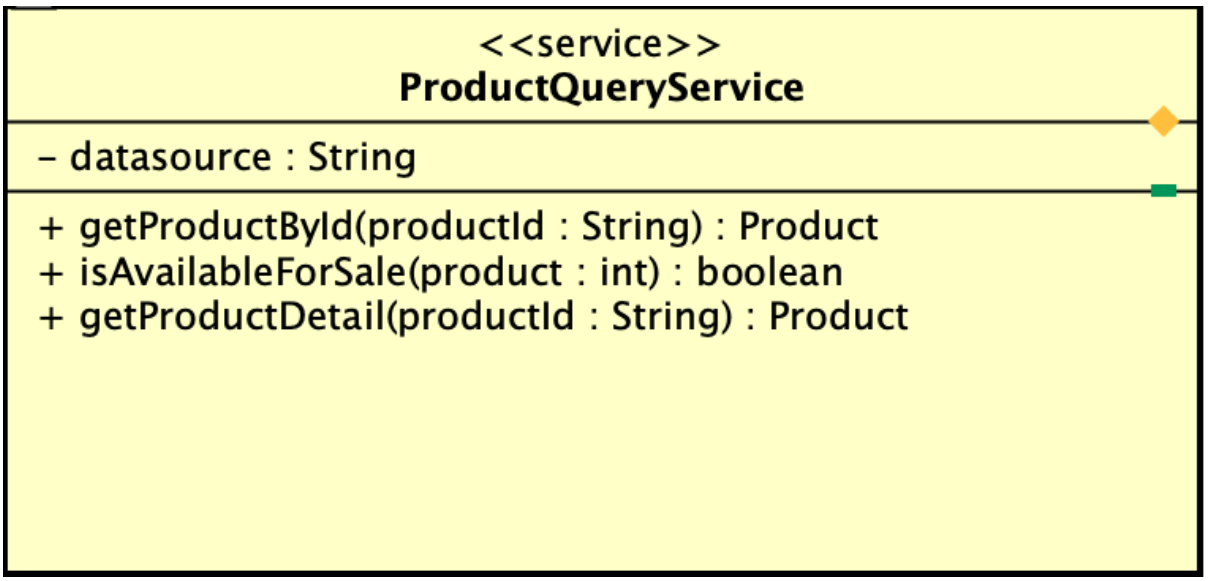


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	dataSource	String	null	Data source for product retrieval

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getProductById(productId: String)	Product	Retrieve product by ID

2	isAvailableForSale(product : Product)	boolean	Check availability
3	getProductDetail(productId : String)	Product	Retrieve full product det

Parameter:

- productId: product identifier.
- product: product object.

Exception:

- IllegalArgumentException if invalid ID
- SystemDataAccessException if data access fails.

Method

- getProductById() retrieves product.
- getProductDetail() returns full detail
- isAvailableForSale() checks status and stock.

Other diagrams

- Not required

17. Design for class “Product”

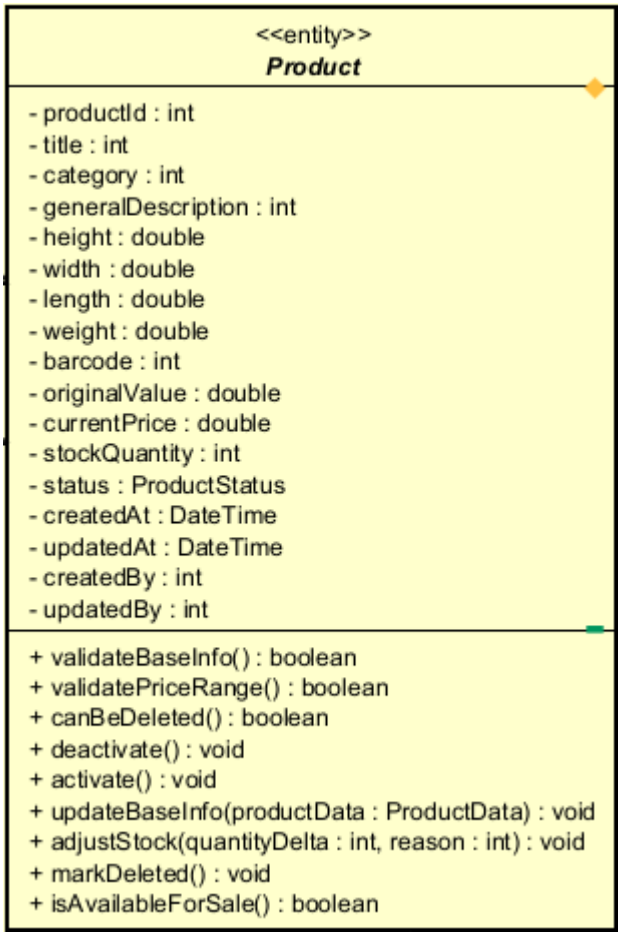


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productId	String	null	Product identifier
2	title	String	""	Product title
3	category	String	""	Category
4	generalDescription	String	""	General description

5	height	double	0.0	Height
6	width	double	0.0	Width
7	length	double	0.0	Length
8	weight	double	0.0	Weight
9	barcode	String	""	Barcode
10	originalValue	double	0.0	Original value excluding VAT
11	currentPrice	double	0.0	Current price excluding VAT
12	stockQuantity	int	0	Stock quantity
13	status	ProductStatus	DRAFT	Product status
14	createdAt	DateTime	now()	Creation time
15	updatedAt	DateTime	now()	Last updated time
16	createdBy	String	null	Manager who created the product

17	updatedBy	String	null	Manager who most recently updated the product

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateBaseInfo()	boolean	Validate the general product information
2	validatePriceRange()	boolean	Check whether currentPrice is within 30%–150% of originalValue
3	canBeDeleted()	boolean	Check whether the product can be physically deleted
4	deactivate()	void	Change the status to deactivated
5	activate()	void	Reactivate the product
6	updateBaseInfo(...)	void	Update general information
7	adjustStock(quantityDelta:int, reason:String)	void	Adjust stock quantity
8	markDeleted()	void	Mark the product as deleted
9	isAvailableForSale()	boolean	Check whether the product is still available for sale

Parameter:

- quantityDelta: 0, amount of stock increase/decrease
- reason: "", reason for stock change
- title/category/generalDescription/...: updated data

Exception:

- InvalidProductInfoException if the general data is invalid
- InvalidPriceRangeException if the price is outside the allowed range
- ProductDeletionNotAllowedException if deletion is requested but not allowed by policy
- InvalidStockAdjustmentException if stock becomes negative or the reason is empty

Method

- validateBaseInfo() checks title, dimensions, weight, barcode, originalValue, and currentPrice.
- validatePriceRange() checks 0.3 * originalValue <= currentPrice <= 1.5 * originalValue.
- canBeDeleted() returns true when stockQuantity == 0.
- deactivate() changes status = DEACTIVATED.
- markDeleted() is only called after passing canBeDeleted().

Other diagrams

- State diagram for Product
- Activity diagram for delete/deactivate decision

18. Design for class “Book”

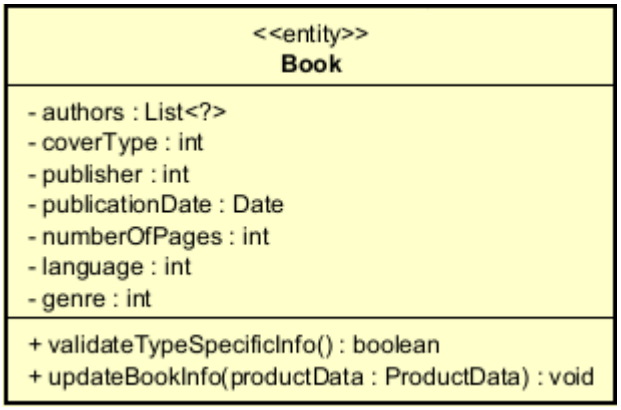


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	authors	List	[]	List of authors
2	coverType	String	""	Paperback/Hardcover
3	publisher	String	""	Publisher
4	publicationDate	Date	null	Publication date
5	numberOfPages	int	0	Number of pages
6	language	String	""	Language
7	genre	String	""	Genre

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateTypeSpecificInfo()	boolean	Validate book-specific information
2	updateBookInfo(...)	void	Update book-specific information

Parameter:

- authors: list of authors
- coverType: cover type
- publisher: publisher

Exception:

- InvalidProductInfoException if authors are missing, publisher is missing, or the publication date is invalid

Method

- Check that authors is not empty, publisher is not empty, and publicationDate is valid.

Other diagrams

- None

19. Design for class “CD”

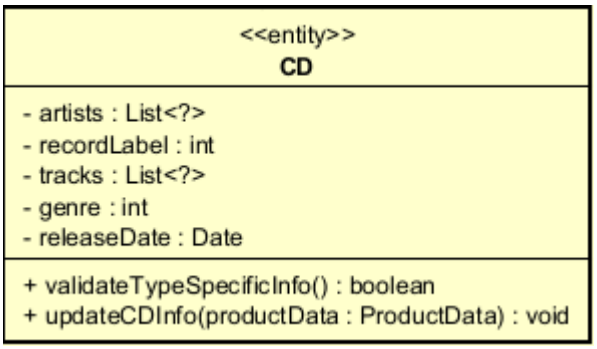


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	artists	List	[]	Artists
2	recordLabel	String	""	Record label
3	tracks	List	[]	List of tracks
4	genre	String	""	Genre

5	releaseDate	Date	null	Release date

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateTypeSpecificInfo()	boolean	Validate CD-specific information
2	updateCDInfo(...)	void	Update CD-specific information

Exception:

- InvalidProductInfoException if artist, recordLabel, or track list is missing

20. Design for class “DVD”

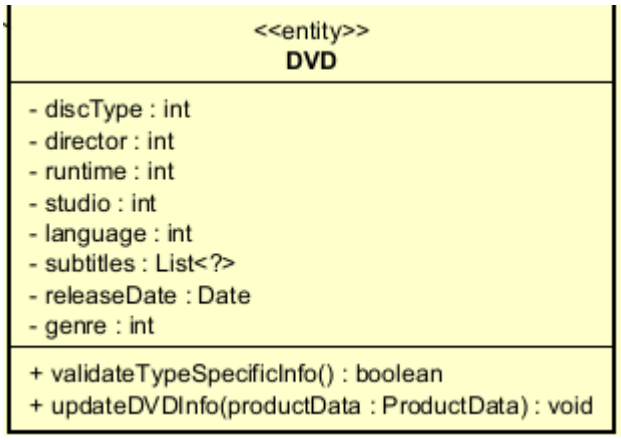


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	discType	String	""	Blu-ray / HD-DVD
2	director	String	""	Director
3	runtime	int	0	Runtime
4	studio	String	""	Studio
5	language	String	""	Language
6	subtitles	List	[]	Subtitles
7	releaseDate	Date	null	Release date
8	genre	String	""	Genre

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateTypeSpecificInfo()	boolean	Validate DVD-specific information
2	updateDVDInfo(...)	void	Update DVD-specific information

21. Design for class “Newspaper”

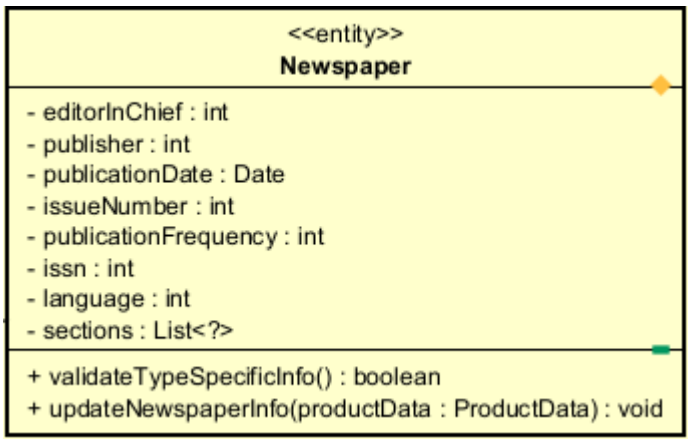


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	editorInChief	String	""	Editor-in-chief
2	publisher	String	""	Publisher
3	publicationDate	Date	null	Publication date
4	issueNumber	String	""	Issue number
5	publicationFrequency	String	""	Publication frequency
6	issn	String	""	ISSN
7	language	String	""	Language
8	sections	List	[]	Sections

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateTypeSpecificInfo()	boolean	Validate newspaper-specific information
2	updateNewspaperInfo(...)	void	Update newspaper-specific information

22. Design for class “ProductHistory”

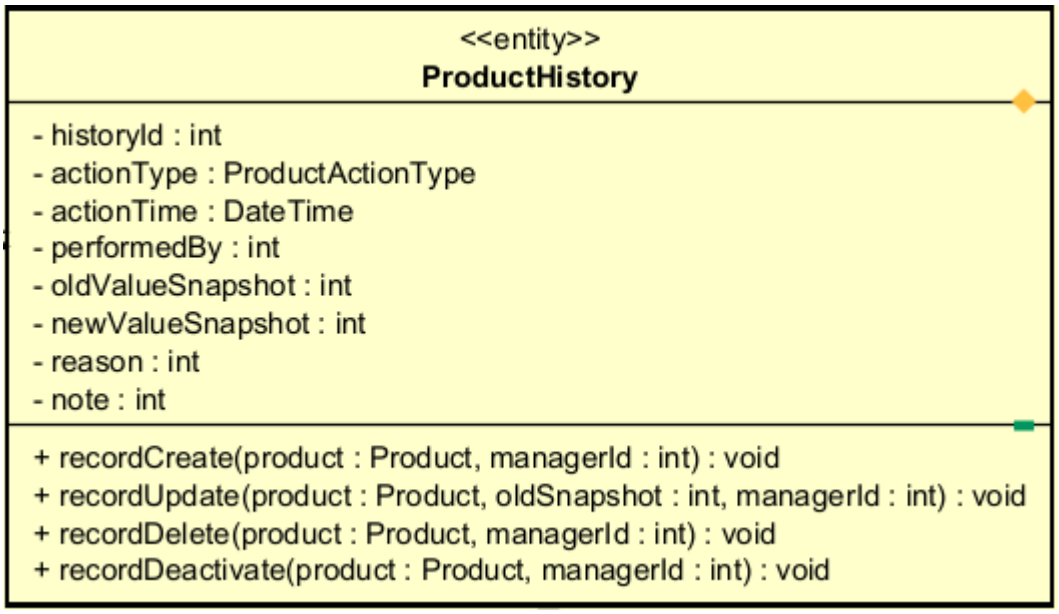


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	historyId	String	null	History identifier
2	productId	String	null	Related product ID

3	actionType	String	""	CREATE / UPDATE / DELETE / DEACTIVATE / STOCK_ADJUST
4	actionTime	DateTime	now()	Action time
5	performedBy	String	null	Person who performed the action
6	oldValueSnapshot	String	null	Snapshot before the change
7	newValueSnapshot	String	null	Snapshot after the change
8	reason	String	""	Reason for the action
9	note	String	""	Additional note

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	recordCreate(...)	void	Record creation history
2	recordUpdate(...)	void	Record update history
3	recordDelete(...)	void	Record deletion history

4	recordDeactivate(...)	void	Record deactivation history
5	recordStockAdjustment(...)	void	Record stock adjustment history

Parameter:

- productId: product ID
- performedBy: manager performing the action
- reason: reason for the change
- oldSnapshot/newSnapshot: data before/after the change

Method

- Each method sets the corresponding actionType, timestamp, actor, and snapshots

23. Design for class “ProductManagementService”

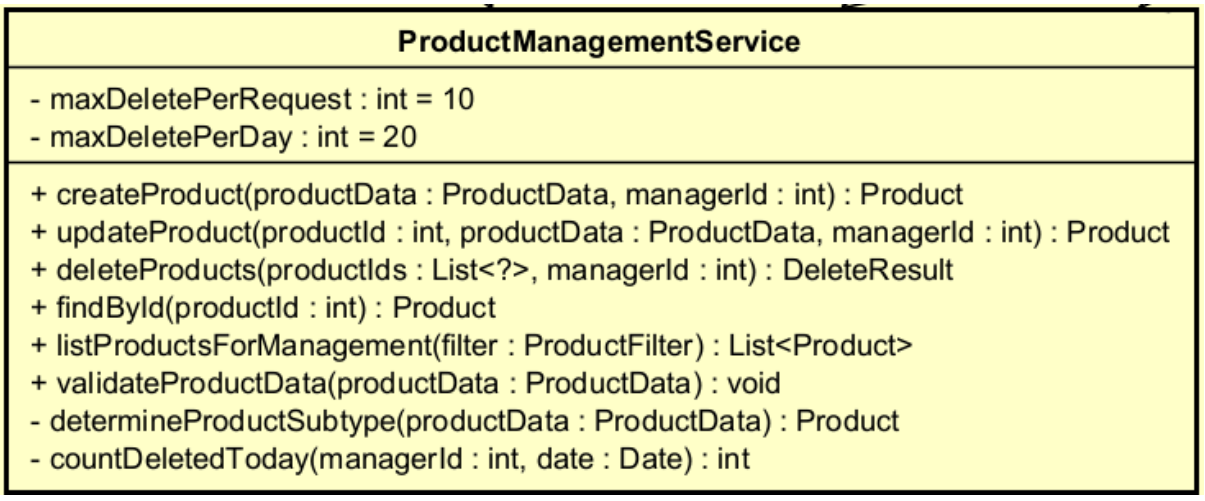


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productRepository	ProductRepository	injected	Access product data

2	validator	ProductValidator	injected	Validate data
3	maxDeletePerRequest	int	10	Maximum deletions per request
4	maxDeletePerDay	int	20	Maximum deletions per day

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	createProduct(productData:ProductData, managerId:String)	Product	Create a new product
2	updateProduct(productId:String, productData:ProductData, managerId:String)	Product	Update a product
3	deleteProducts(productIds:List, managerId:String)	DeleteResult	Delete/deactivate a list of products
4	findById(productId:String)	Product	Find a product by ID
5	listProductsForManagement(filter:ProductFilter)	List	Retrieve product list for management
6	validateProductData(productData:ProductData)	void	Validate input data

7	determineProductSubtype(productData:ProductData)	Product	Instantiate the correct subtype
8	countDeletedToday(managerId:String, date:Date)	int	Count how many products were deleted today

Parameter:

- productData: DTO containing data entered from the form
- productIds: list of selected product IDs
- managerId: person performing the action
- filter: list filtering condition

Exception:

- InvalidProductInfoException if the form data is invalid
- ProductNotFoundException if the product cannot be found
- DeletionLimitExceededException if it exceeds 10 per request or 20 per day
- InvalidPriceRangeException if the price is invalid
- ProductDeletionNotAllowedException if the deletion logic is invalid

Method

- createProduct():
    - validate data
    - instantiate the appropriate subtype (Book/CD/DVD/Newspaper)
    - set status to ACTIVE
    - save
- updateProduct():
    - load product
    - validate
    - update common + type-specific information
    - save
- deleteProducts():
    - check selected quantity <= 10
    - check total number deleted today <= 20
    - for each product:
        - if stockQuantity == 0 → delete/mark deleted
        - otherwise → deactivate
    - return DeleteResult

24. Design for class “ProductHistoryService”

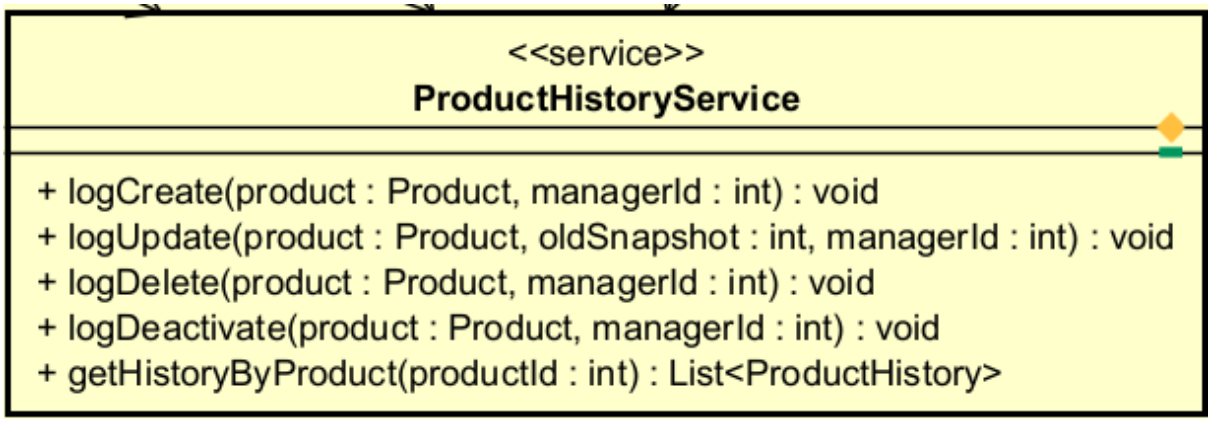


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	historyRepository	ProductHistoryRepository	injected	Access history data

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	logCreate(product:Product, managerId:String)	void	Log create history
2	logUpdate(product:Product, oldSnapshot:String, managerId:String)	void	Log update history
3	logDelete(product:Product, managerId:String)	void	Log delete history
4	logDeactivate(product:Product, managerId:String)	void	Log deactivate history

5	logStockAdjustment(product:Product, reason:String, managerId:String)	void	Log stock adjustment history
6	getHistoryByProduct(productId:String)	List	Query history

Method

- Create a ProductHistory object and save it to the repository according to the corresponding action

25. Design for class “CreateProductController”

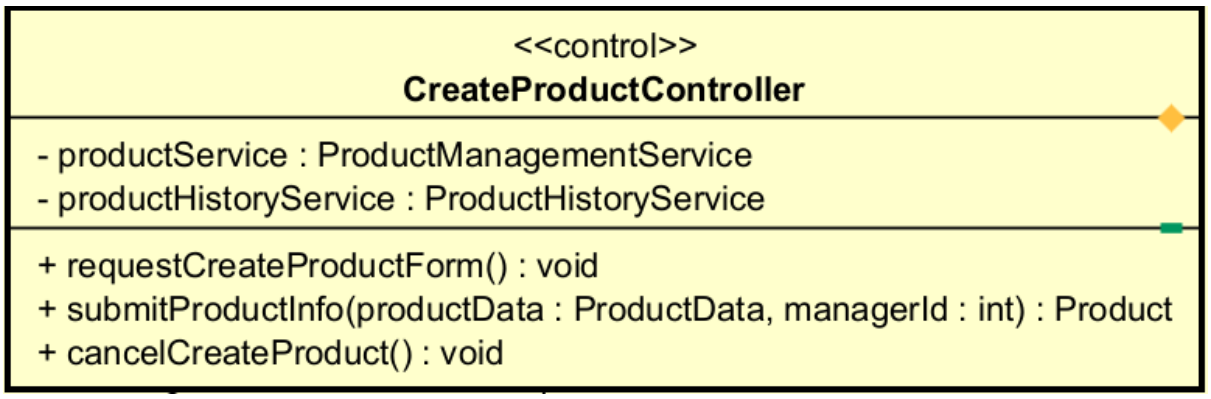


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productService	ProductService	injected	Product creation service
2	productHistoryService	ProductHistoryService	injected	History logging service

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestCreateProductForm()	void	Request to open the creation screen
2	submitProductInfo(productData:ProductData, managerId:String)	Product	Receive and process the create request
3	cancelCreateProduct()	void	Cancel the creation operation

Method

- submitProductInfo() calls productService.createProduct(), then productHistoryService.logCreate().

26. Design for class “UpdateProductController”

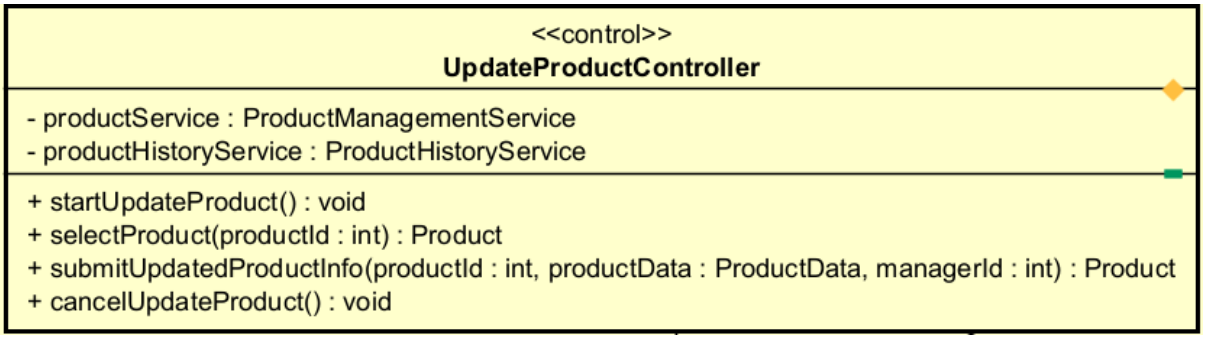


Table 1. Example of attribute design

#	Name	Data type	Default value	Description

1	productService	ProductService	injected	Update service
2	productHistoryService	ProductHistoryService	injected	History logging

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	startUpdateProduct()	void	Start the update use case
2	selectProduct(productId:String)	Product	Select the product to update
3	submitUpdatedProductInfo(productId:String, productData:ProductData, managerId:String)	Product	Process the update request
4	cancelUpdateProduct()	void	Cancel update

Method

- submitUpdatedProductInfo() loads the old product, snapshots the old value, performs update, and logs history

27. Design for class “DeleteProductController”

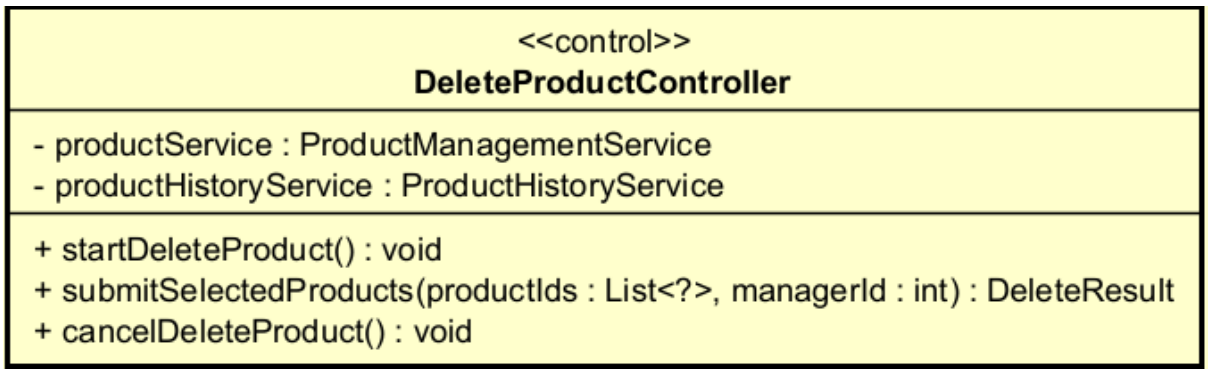


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productService	ProductService	injected	Delete/deactivate service
2	productHistoryService	ProductHistoryService	injected	History logging

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	startDeleteProduct()	void	Start the delete use case
2	submitSelectedProducts(productIds:List, managerId:String)	DeleteResult	Process the selected product list
3	cancelDeleteProduct()	void	Cancel deletion

Method

- Call productService.deleteProducts()
- For each item in the result:
  - if physically deleted → logDelete()
  - if deactivated → logDeactivate()
  -

28. Design for class “CreateProductScreen”

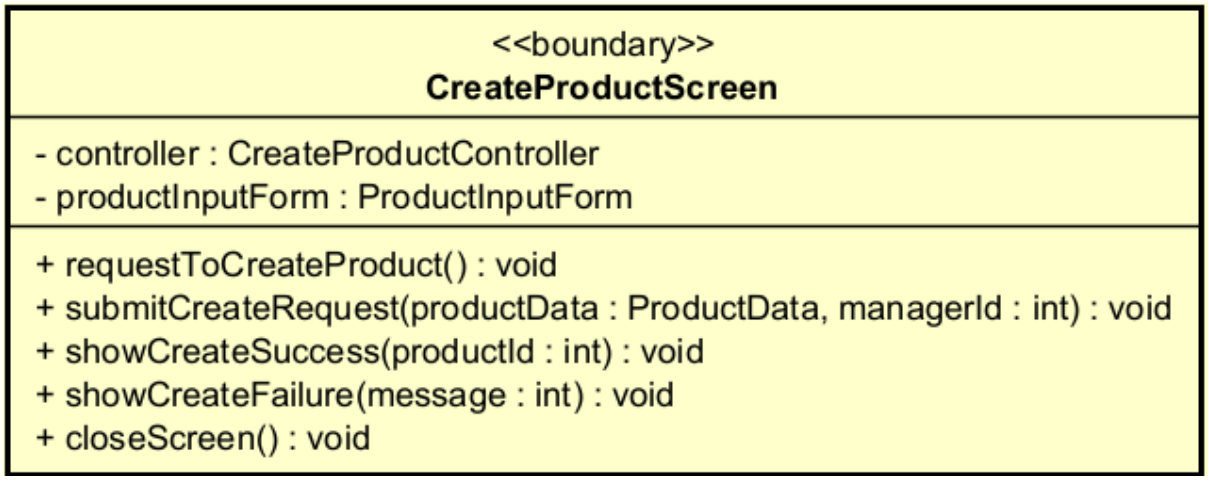


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	controller	CreateProduct Controller	injected	Coordinating controller
2	productInputForm	ProductInputF orm	null	Data input form

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToCreateProduct()	void	Open the product creation form

2	showCreateSuccess(productId:String)	void	Display successful creation
3	showCreateFailure(message:String)	void	Display creation error
4	closeScreen()	void	Close the screen

29. Design for class “ProductInputForm”

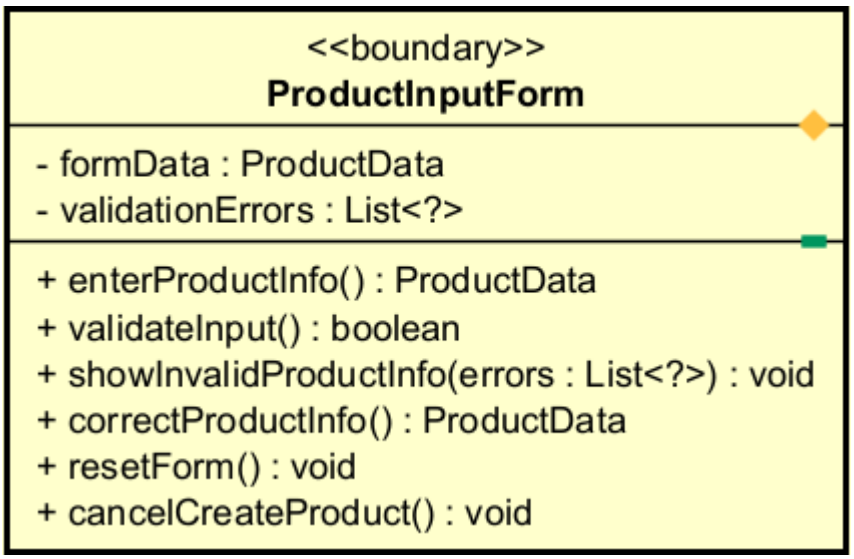


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	formData	ProductData	empty	Data entered by the user
2	validationErrors	List	[]	List of validation errors

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	enterProductInfo()	ProductData	Enter product data
2	validateInput()	boolean	Validate input on the form side
3	showInvalidProductInfo(errors:List)	void	Display errors
4	correctProductInfo()	ProductData	Allow re-entry
5	resetForm()	void	Reset form
6	cancelCreateProduct()	void	Cancel operation

30. Design for class “DeleteProductScreen”

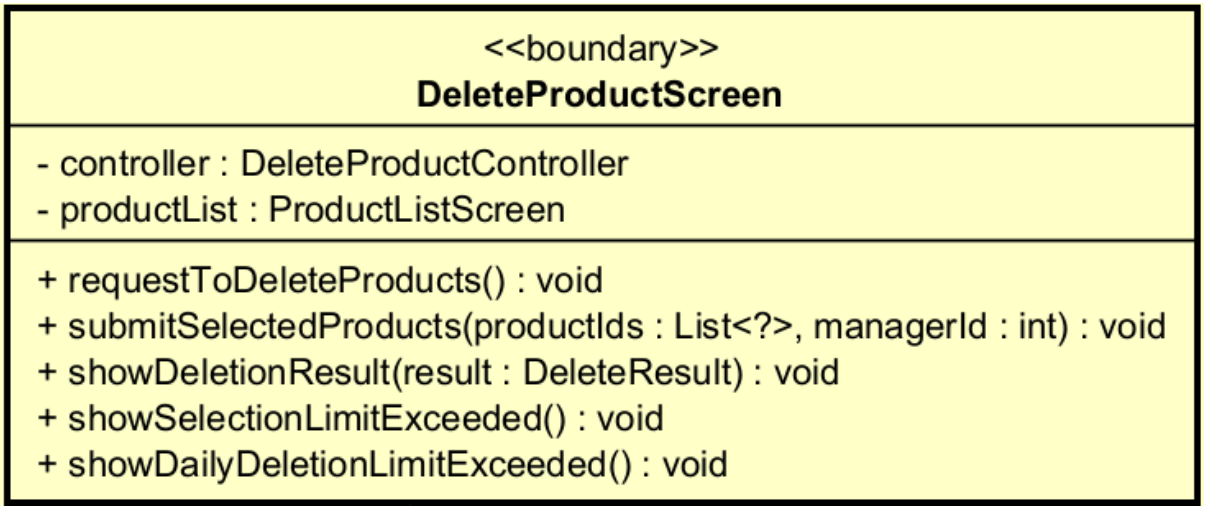


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	controller	DeleteProduct Controller	injected	Delete controller
2	productList	ProductList	null	Product list for selection

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToDeleteProducts()	void	Open the delete function
2	confirmDelete(productIds:List)	void	Confirm deletion
3	showDeletionResult(result: DeleteResult)	void	Display result
4	showSelectionLimitExceeded()	void	Notify that 10 items/request is exceeded
5	showDailyDeletionLimitExceeded()	void	Notify that 20 items/day is exceeded

31. Design for class “ProductUpdateScreen”

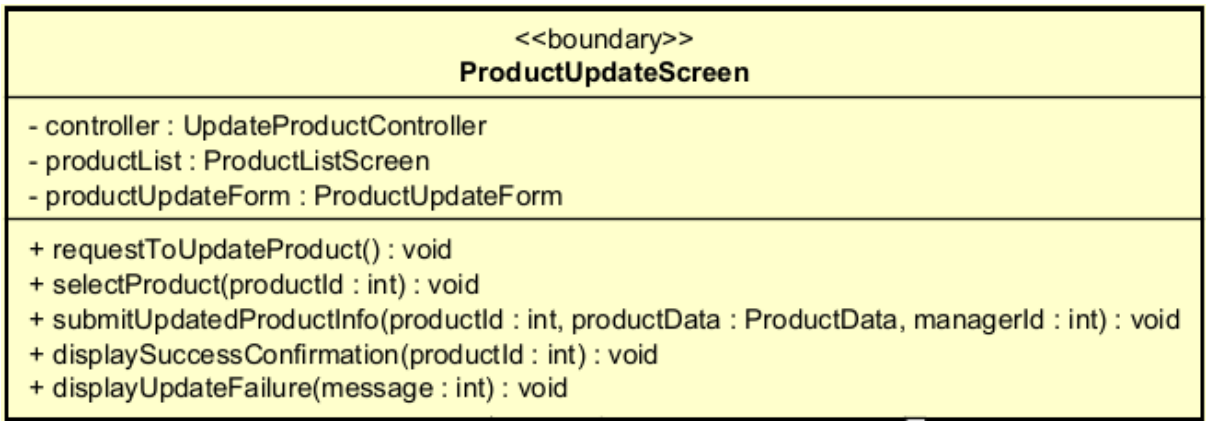


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	controller	UpdateProductController	injected	Update controller
2	productList	ProductList	null	List for selecting products
3	productUpdateForm	ProductUpdateForm	null	Data editing form

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToUpdateProduct()	void	Start the update use case

2	showProductList(products: List)	void	Display the list
3	selectProduct(productId:String)	void	Select product
4	displaySuccessConfirmation(productId:String)	void	Notify successful update
5	displayUpdateFailure(message:String)	void	Notify error

32. Design for class “ProductUpdateForm”

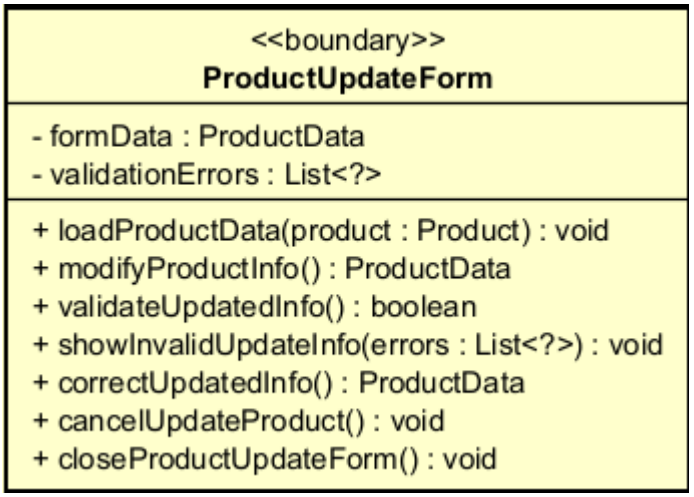


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	formData	ProductData	empty	Updated data

2	validationErrors	List	[]	Validation errors

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadProductData(product:Product)	void	Load current product data into the form
2	modifyProductInfo()	ProductData	Modify information
3	validateUpdatedInfo()	boolean	Validate updated data
4	showInvalidUpdateInfo(errors:List)	void	Display errors
5	correctUpdatedInfo()	ProductData	Allow correction
6	cancelUpdateProduct()	void	Cancel update
7	closeProductUpdateForm()	void	Close the form

33. Design for class “ProductListScreen”

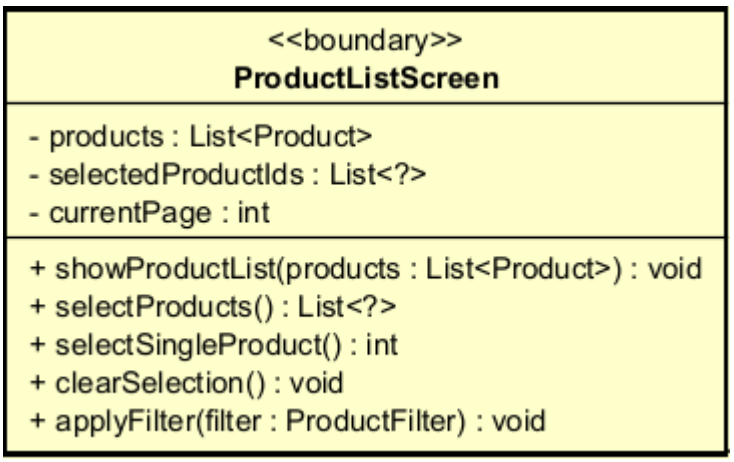


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	products	List	[]	Product list
2	selectedProductIds	List	[]	List of selected IDs
3	filter	ProductFilter	null	Filter
4	currentPage	int	1	Current page

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	showProductList(products: List)	void	Display the list

2	selectProducts()	List	Select multiple products
3	selectSingleProduct()	String	Select one product for update
4	clearSelection()	void	Clear selection
5	applyFilter(filter:ProductFilter)	void	Apply filter

34. Design for class “PaymentScreen”

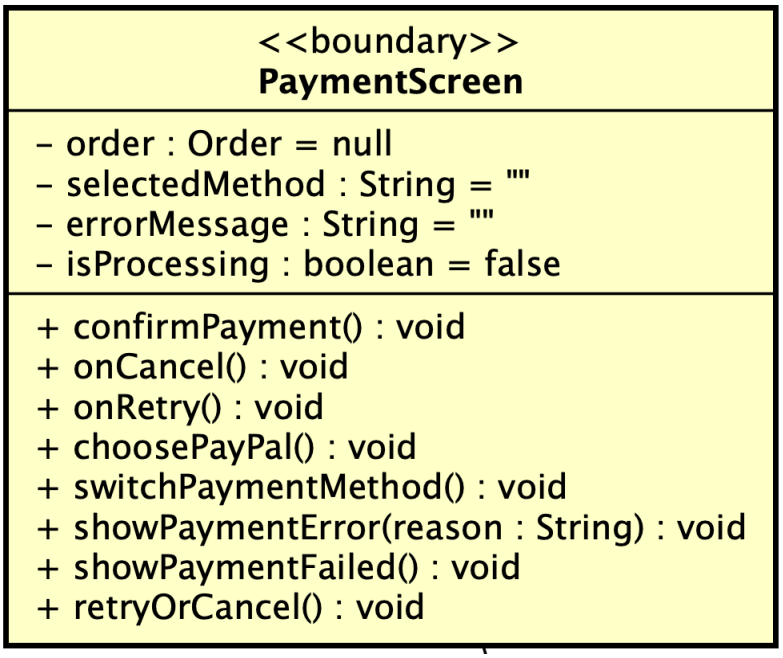


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	order	Order	null	current order
2	selectedMethod	string	""	selected payment method
3	errorMessage	string	""	error message

4	isProcessing	boolean	false	loading state

Table 2. Example of operation design

#	Name	Return type	Description
1	confirmPayment()	void	trigger payment
2	onCancel()	void	cancel payment
3	onRetry()	void	retry payment
4	choosePayPal()	void	select PayPal
5	switchPaymentMethod()	void	switch method
6	showPaymentError(reason :String)	void	show error
7	showPaymentFailed()	void	show failure UI
8	retryOrCancel()	void	user decision

Parameter:

- reason: error message

Method

- call PaymentController.payOrder(order)
- handle UI state (loading/error)

35. Design for class “PaymentController”

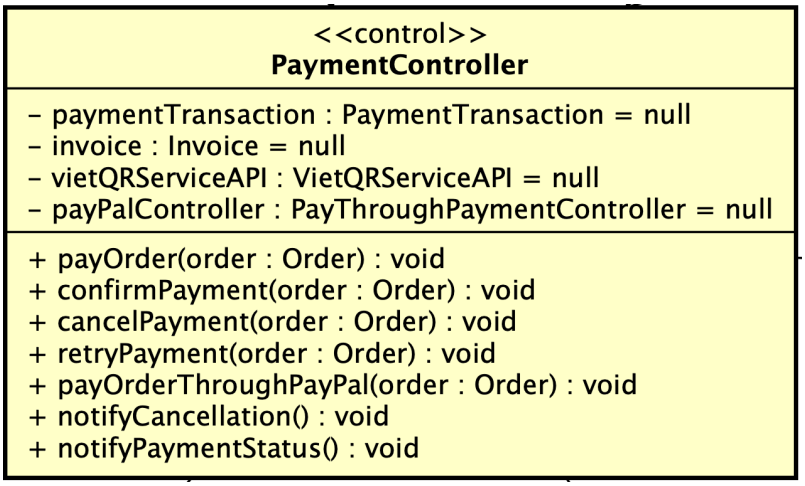


Table 1. Example of attribute design

#	Name	Data type	Default	Description
1	paymentTransaction	PaymentTrans	null	current transaction

		action		
2	invoice	Invoice	null	invoice object
3	vietQRServiceAPI	VietQRService API	null	external API
4	payPalController	PayThroughPaymentController	null	PayPal flow

Table 2. Example of operation design

#	Name	Return type	Description
1	payOrder(order:Order)	void	start payment
2	confirmPayment(order:Order)	void	confirm payment
3	cancelPayment(order:Order)	void	cancel payment
4	retryPayment(order:Order)	void	retry flow
5	payOrderThroughPayPal(order:Order)	void	PayPal flow
6	notifyCancellation()	void	notify cancel
7	notifyPaymentStatus()	void	notify result

Parameter:

- order: Order (object, pass by reference)

Exception:

- PaymentFailedException if payment fails

Method

- create PaymentTransaction
- call appropriate flow:
    - PayPal -> payOrderThroughPayPal
    - QR -> call QR controller
- update transaction state

36. Design for class “PayByQRCodeController”

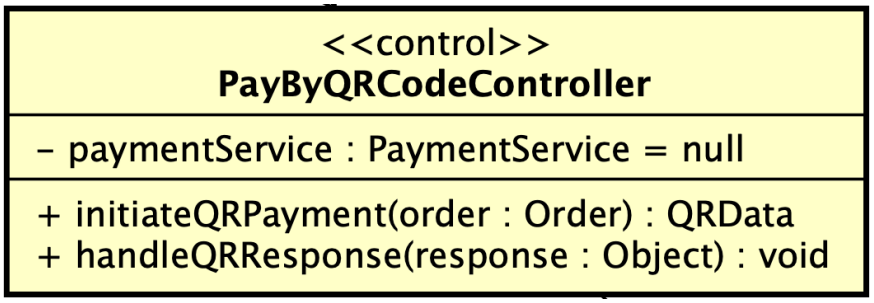


Table 1. Example of attribute design

#	Name	Data type	Default	Description
1	paymentService	PaymentService	null	business logic

Table 2. Example of operation design

#	Name	Return type	Description
1	initiateQRPayment(order:Order)	QRData	create QR
2	handleQRResponse(response)	void	process response

Parameter:

- call paymentService.createQRPayment(order)
- return QR data for UI

37. Design for class “PaymentService”

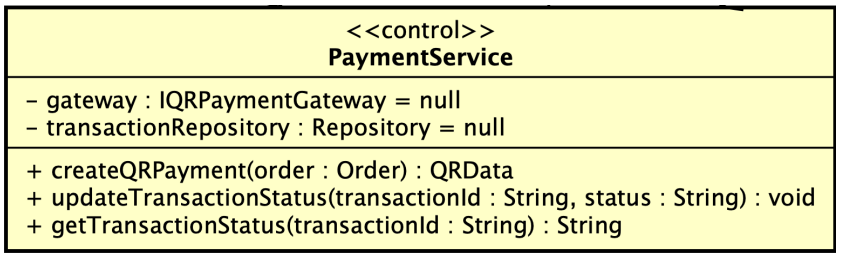


Table 1. Example of attribute design

#	Name	Data type	Default	Description
1	gateway	IQRPaymentGateway	null	abstraction
2	transactionRepository	Repository	null	DB access

Table 2. Example of operation design

#	Name	Return type	Description
1	createQRPayment(order:Order)	QRData	create transaction
2	updateTransactionStatus(transactionId:String, status:String)	void	update
3	getTransactionStatus(transactionId:String)	String	get status

Exception:

- PaymentFailedException if gateway fails

Method

- create PaymentTransaction (PENDING)
- call gateway.generateQRCode()
- save transaction
- return QR

38. Design for class “PaymentTransaction”

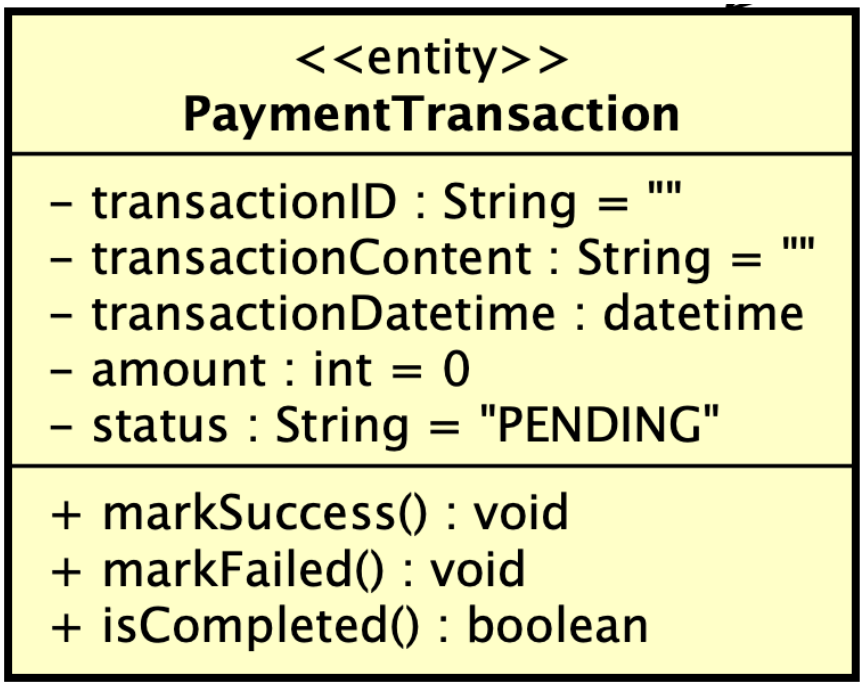


Table 1. Example of attribute design

#	Name	Data type	Default	Description

1	transactionID	String	""	unique id
2	transactionContent	String	""	content
3	transactionDatetime	datetime	now	time
4	amount	int	0	amount
5	status	enum	PENDING	state

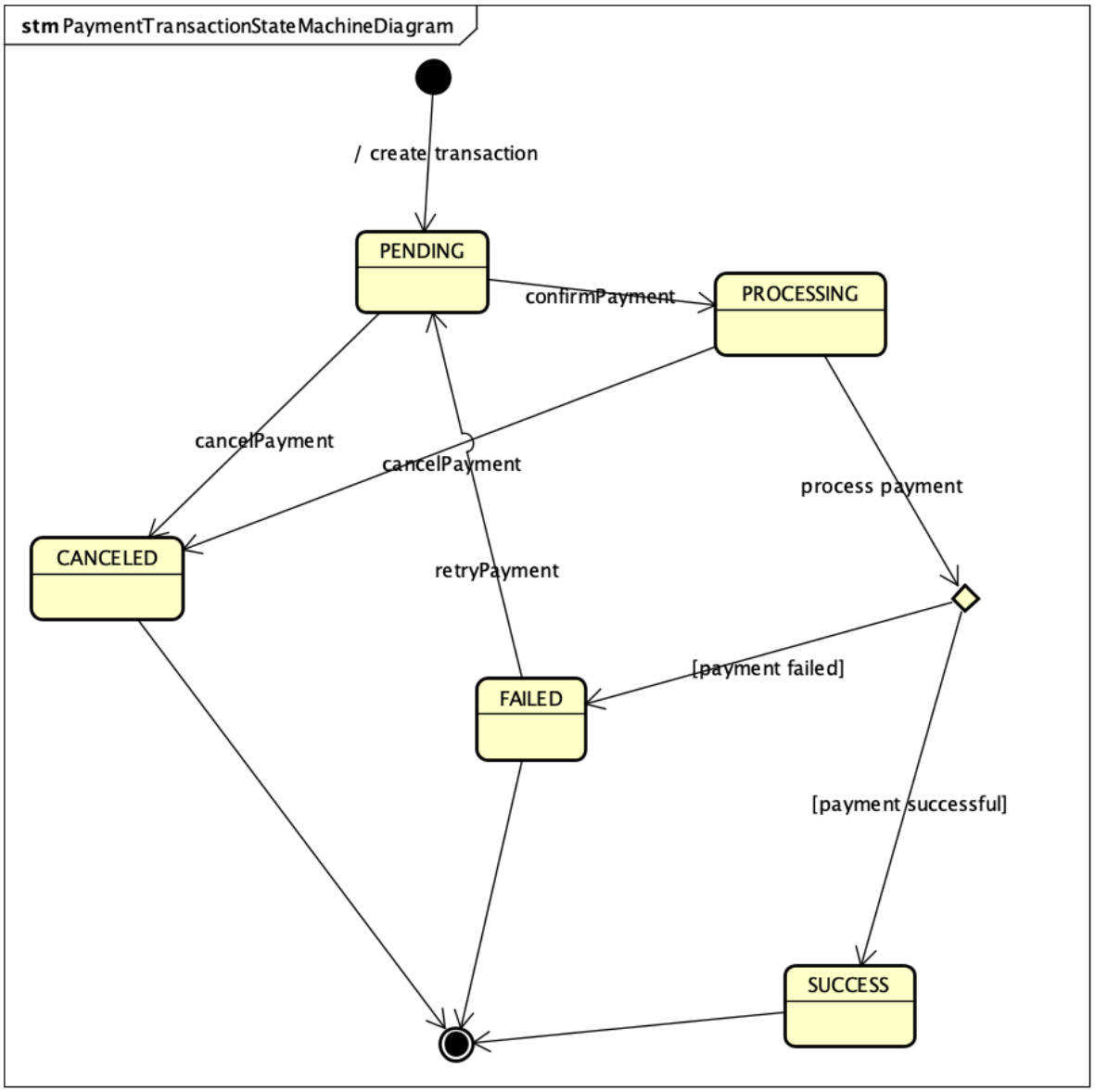
Table 2. Example of operation design

#	Name	Return type	Description
1	markSuccess()	void	set SUCCESS
2	markFailed()	void	set FAILED
3	isCompleted()	boolean	check done

Method

- change status
- persist to DB

Other diagram



39. Design for class “VietQRServiceAPI”

<<boundary>> **VietQRServiceAPI**
- baseUrl : String = "https://api.vietqr.vn" - clientId : String = "" - apiKey : String = "" - accessToken : String = ""
+ getAccessToken(username : String, password : String) : TokenResponse + generateQRCode(request : VietQRRequest) : VietQRResponse + buildQRRequest(order : Order) : VietQRRequest

Table 1. Example of attribute design

#	Name	Data type	Default value	Description

1	baseUrl	String	"https://api.vietqr.vn"	API endpoint
2	clientId	String	""	authentication id
3	apiKey	String	""	authentication key
4	accessToken	String	""	bearer token

Table 2. Example of operation design

#	Name	Return type	Description
1	getAccessToken(username :String, password:String)	TokenResponse	get token
2	generateQRCode(request: VietQRRequest)	VietQRRespons e	create QR
3	buildQRRequest(order:Ord er)	VietQRRequest	map order → request

Parameter:

- username/password: credential
- request contains:
    - accountNo
    - accountName
    - acqId
    - amount
    - addInfo

Exception:

- PaymentFailedException if API call fails

Method

- getAccessToken():
    - POST /get-token
    - lưu bearer token
- generateQRCode():
    - attach header: Authorization: Bearer <token>
    - POST /generate
    - return QR data
- buildQRRequest():
    - map:
        - order.totalAmount -> amount
        - orderId -> addInfo

40. Design for class “PaymentResultScreen”

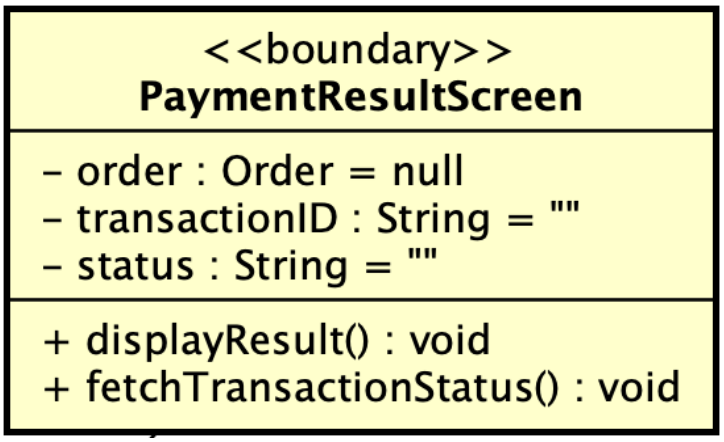


Table 1. Example of attribute design

#	Name	Data type	Default	Description
1	order	Order	null	order
2	transactionID	String	""	id
3	status	String	""	result

Table 2. Example of operation design

#	Name	Return type	Description
1	displayResult()	void	show result
2	fetchTransactionStatus()	void	call backend

41. Design for class “VietQRCallbackHandler”

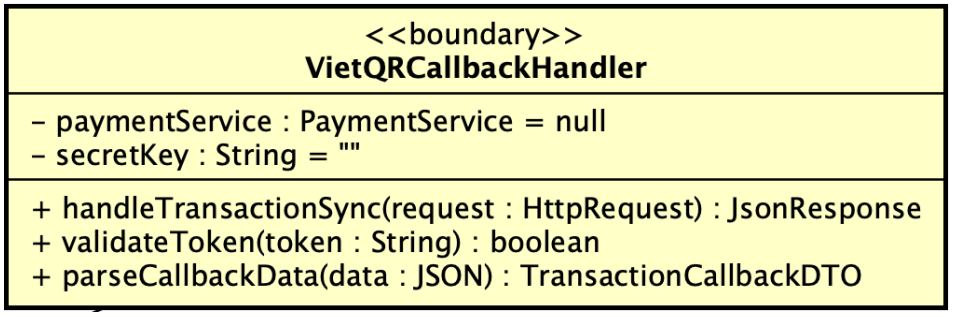


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	paymentService	PaymentService	null	business service

2	secretKey	String	""	validate token

Table 2. Example of operation design

#	Name	Return type	Description
1	handleTransactionSync(request:HttpRequest)	JsonResponse	receive callback
2	validateToken(token:String)	boolean	validate bearer token
3	parseCallbackData(data:JSON)	TransactionCallbackDTO	parse request

Method

- parse request
- call updateTransactionStatus()
- return success response

42. Design for class “IQRPaymentGateway”

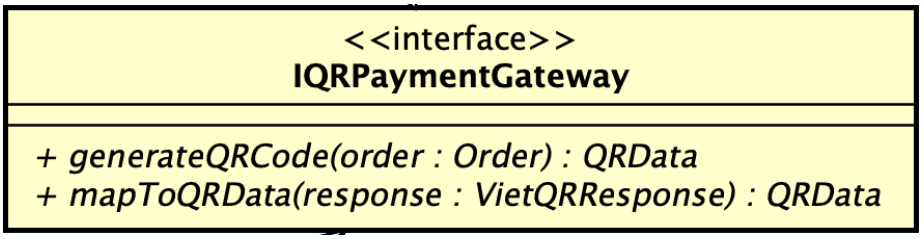


Table 2. Example of operation design

#	Name	Return type	Description
1	generateQRCode(order:Order)	QRData	create QR
2	mapToQRData(response:VietQRResponse)	QRData	map API response

Parameter:

- order: Order object

Method:

- define contract giữa service và gateway

43. Design for class “VietQRGateway”

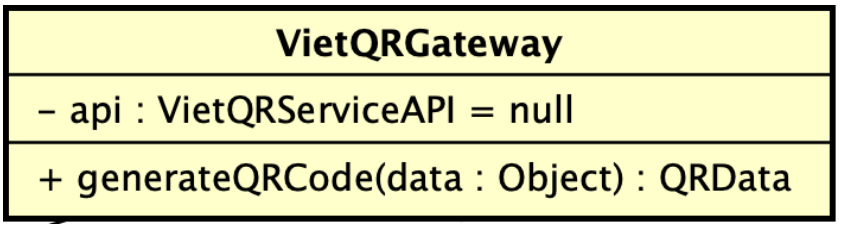


Table 1. Example of attribute design

#	Name	Data type	Default	Description
1	api	VietQRService API	null	external API

Table 2. Example of operation design

#	Name	Return type	Description
1	generateQRCode(data)	QRData	implement

Method

- call API
- map response → QRData

44. Design for class “QRPaymentScreen”

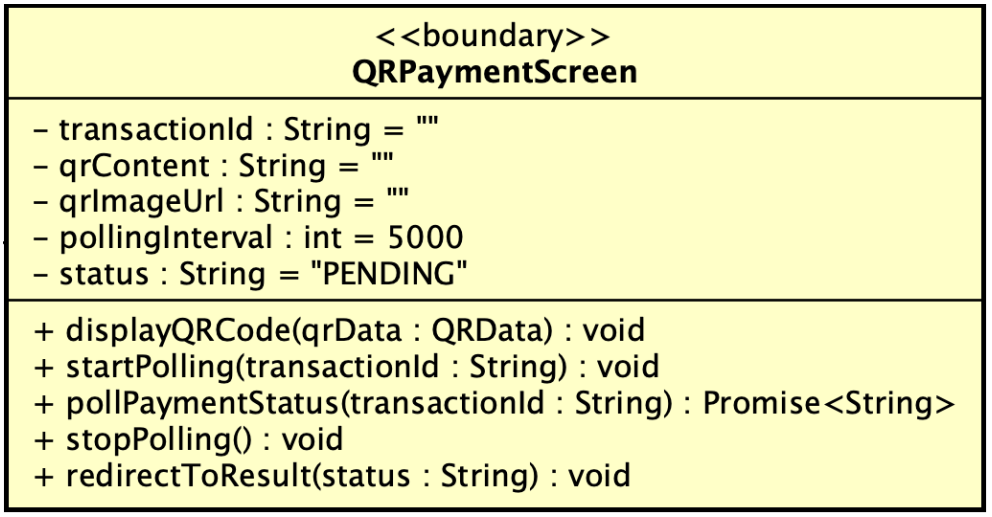


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	""	payment transaction

				id
2	qrContent	string	""	QR content string
3	qrImageUrl	string	""	QR image URL
4	pollingInterval	number	5000	polling interval (ms)
5	status	string	"PENDING"	transaction status

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayQRCode(qrData:QRData)	void	render QR code
2	startPolling(transactionId:String)	void	start polling status
3	pollPaymentStatus(transactionId:String)	Promise<String>	call backend API
4	stopPolling()	void	stop polling
5	redirectToResult(status:String)	void	navigate result screen

Parameter:

- qrData: contains qrContent, qrImageUrl
- transactionId: unique transaction

Method

- display QR từ generateQRCode API
- polling /payment/status
- nếu status != PENDING → redirect

45. Design for class “PayPalBoundary”

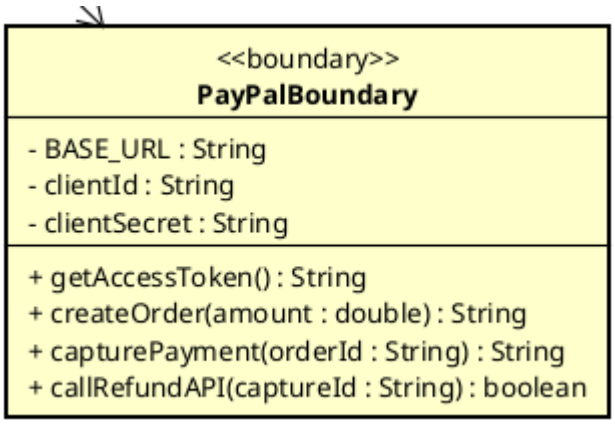


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	BASE_URL	String	"https://api-m.sandbox.paypal.com"	The base URL for the PayPal REST API sandbox.
2	clientID	String	“”	Unique identifier for the application provided by PayPal.
3	clientSecret	String	“”	Private key used for OAuth 2.0 authentication

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	String	Requests a Bearer token to authorize subsequent API calls
2	createOrder	String	Creates a PayPal order and returns the unique Order ID.
3	capturePayment	Object	Finalizes the transaction after the buyer's approval.
4	callRefundAPI	Object	Excutes a refund request for a specific transaction ID.

Parameter:

- amount: double - Total payment amount (including VAT and shipping).
- orderId: String - The ID returned by PayPal after order creation.

Exception:

- PayPalAuthException if: Authentication fails due to invalid credentials.
- PayPalNetworkException if: There is a connection timeout or server error.

Method

- Uses HTTP POST requests to interact with PayPal's V2 Orders and Refund APIs.
- Handles JSON responses to extract necessary transaction data.

46. Design for class “PayThroughPayPalController”

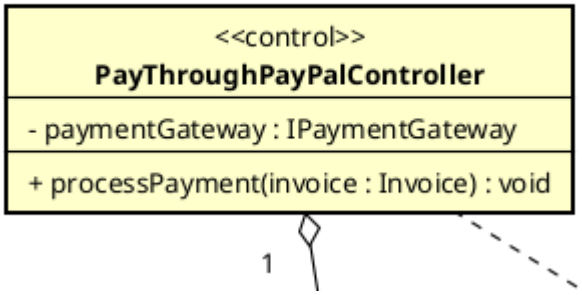


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paymentGateway	IPatmentGateway	null	Reference to the PayPal gateway implementation.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	processPayment	void	Coodinates the end-to-end payment workflow.

Parameter:

- invoice: Invoice - Contains billing information and shipping fees.

Exception:

- PaymentCanceledException if: The user cancels the transaction manually.

Method

- Extracts final price from the invoice.
- Invokes paymentGateway.payOrder() and logs the result into a PaymentTransaction.

47. Design for class “IPaymentGateway”

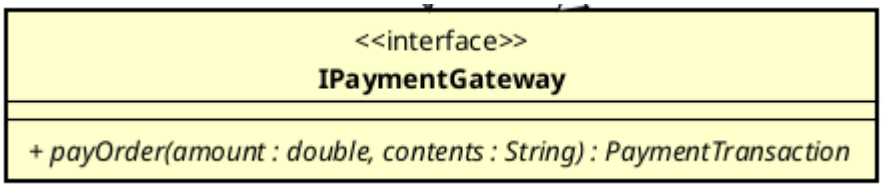


Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	payOrder	PaymentTransaction	Interface for processing a payment order.

Parameter:

- amount: double - The total amount to be paid.
- contents: String - Transaction description.

48. Design for class “IRefundGateway”

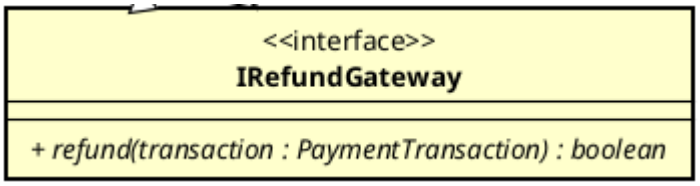


Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	refund	boolean	Abstract method to process an automated refund for a previous transaction.

Parameter:

- transaction: PaymentTransaction - The original successful transaction to be refunded.

49. Design for class “PayPalGateway”

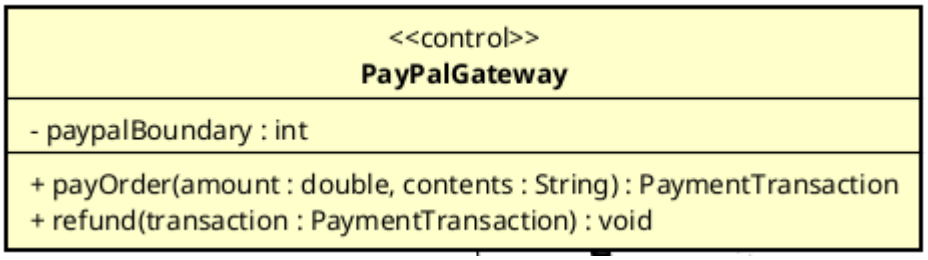


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	paypalBoundary	PayPalBoundary	new PayPalBoundary()	Direct interface to PayPal API communication.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	payOrder	PaymentTransaction	Processes a payment via the PayPal API boundary.
2	refund	boolean	Processes a refund via the PayPal API boundary.

Method

- Wraps business data into API-compatible objects.
- Returns a PaymentTransaction entity containing the PayPal reference ID.

50. Design for class “RefundService”

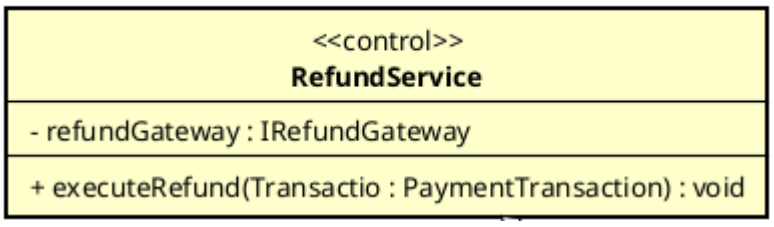


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	refundGateway	IRefundGateway	null	The specific gateway used for the refund

				process.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	executeRefund	void	Manages automated or manual refund logic.

Method

- Identifies the payment method.
- Calls refundGateway.refund() for PayPal transactions.
- Notifies the Product Manager for manual refund if the method is VietQR.